Lab 9: Password Cracking

Course: Vulnerability Assessment and Testing **Lab Type:** In-Class + Take-Home **Duration:** 2.5-3 hours (In-Class: ~1.5 hours, Take-Home: ~1.5 hours) **Points:** 100

Overview

Every credential you extracted in Lab 8 was stored in plaintext. That is rare in modern applications. Most systems store passwords as cryptographic hashes -- one-way transformations that cannot be reversed mathematically. When attackers compromise a database, they get hash values, not passwords. The next step is offline password cracking: systematically hashing candidate passwords and comparing the results to the stolen hashes.

This lab introduces two industry-standard cracking tools -- John the Ripper and Hashcat -- and progresses from basic dictionary attacks through rule-based techniques to a realistic enterprise breach scenario. You will crack password hashes of increasing difficulty and write a professional password audit report documenting your findings.

The skills you learn here apply directly to penetration testing engagements, where a password audit section is often the most actionable part of the final report.

Objectives

By the end of this lab, you will be able to:

1. Explain how password hashing works and why it matters for security
2. Identify common hash types by their format and length
3. Perform dictionary attacks using John the Ripper and Hashcat
4. Use rule-based and mask attacks to crack passwords that resist dictionary attacks
5. Analyze cracking results to identify password policy weaknesses
6. Write a professional password audit report with actionable recommendations

---

Legal and Ethical Notice

Password cracking tools are powerful and legal to use on hashes you are authorized to test. In this lab, all hash files are provided by the instructor for educational purposes. You are authorized to crack ONLY:

- Hash files provided in this lab
- Hashes you generate yourself for testing
- Hashes from systems where you have explicit written permission

Do not use these tools against any system, database, or hash file without authorization. Unauthorized password cracking violates federal computer fraud laws and university policy.

Pre-Lab Preparation

Ensure the following before starting:

- [ ] Kali Linux VM running
- [ ] Verify John the Ripper is installed: `john --help`
- [ ] Verify Hashcat is installed: `hashcat --help`
- [ ] Prepare the rockyou.txt wordlist (see Section 3.1)

Section 1: How Passwords Are Stored (15 minutes)

Before cracking passwords, you need to understand how they are stored and why hashing exists.

1.1 -- Hashing Fundamentals

A hash function takes an input of any length and produces a fixed-length output. The same input always produces the same output, but you cannot work backwards from the output to recover the input.

Try it yourself. Hash the word "password" with two different algorithms:

```
echo -n "password" | md5sum
echo -n "password" | sha256sum
```

Notice:

- MD5 produces a 32-character hexadecimal string (128 bits)
- SHA-256 produces a 64-character hexadecimal string (256 bits)
- Both are deterministic -- run them again and you get identical output

Now hash "Password" (capital P):

```
echo -n "Password" | md5sum
```

The output is completely different. Even a single character change produces a totally different hash. This is called the **avalanche effect**.

1.2 -- Why Hashing Matters

When a web application stores your password, it should store the hash, not the plaintext:

```
User registers with password "Summer2025!"
  -> Application computes: SHA256("Summer2025!") = 541964...
  -> Database stores: username=jsmith, password_hash=541964...

User logs in with "Summer2025!"
  -> Application computes: SHA256("Summer2025!") = 541964...
  -> Compares to stored hash: MATCH -> access granted
```

If the database is stolen, the attacker gets hashes, not passwords. But the attacker can compute hashes too -- that is what password cracking does.

1.3 -- Salts

A **salt** is a random value added to the password before hashing. Each user gets a unique salt, so even if two users have the same password, their hashes are different:

```
User A: SHA256("randomsalt1" + "password") = abc123...
User B: SHA256("randomsalt2" + "password") = def456...
```

Salts prevent precomputed attacks (rainbow tables) and force attackers to crack each hash individually. Unsalted hashes are much easier to crack.

1.4 -- Common Hash Types

Hash Type	Length	Example Use	Cracking Speed
MD5	32 hex chars	Legacy web apps, file checksums	Very fast (~billions/sec on GPU)
SHA-256	64 hex chars	Modern web apps (unsalted = weak)	Fast (~billions/sec on GPU)
NTLM	32 hex chars	Windows/Active Directory	Very fast (~billions/sec on GPU)
SHA-512 crypt (6)	Starts with `$6$`	Linux /etc/shadow	Slow (~millions/sec on GPU)
bcrypt ($2b$)	Starts with `$2b$`	Modern secure apps	Very slow (~thousands/sec on GPU)

> **Q1 (3 points):** Hash the word "cybersecurity" using both `md5sum` and `sha256sum`. Paste both hash outputs. Then hash "Cybersecurity" (capital C) with `md5sum`. Is the output the same or different? In one sentence, explain why.

Section 2: Identifying Hash Types (10 minutes)

When you encounter a stolen hash, your first step is identifying its type. This determines which tool settings to use.

2.1 -- Identification by Format

The most reliable way to identify hashes is by their format:

Clue	Hash Type
32 hex characters, no prefix	MD5 or NTLM (context determines which)
64 hex characters, no prefix	SHA-256
128 hex characters, no prefix	SHA-512
Starts with `$6$` + salt + `$` + hash	SHA-512 crypt (Linux shadow)
Starts with `$2b$` or `$2a$`	bcrypt
Starts with `$1$`	MD5 crypt (legacy Linux shadow)
Format: `user:RID:LM:NTLM:::`	Windows NTDS.dit / SAM dump

2.2 -- Practice Identification

Look at each hash below and determine its type based on format:

```
Hash A: 5f4dcc3b5aa765d61d8327deb882cf99
Hash B: $2b$12$LJ3m4ys3Lg2VbEphMOoMi.eB8UBKEFNBqf2c.RJ1Y.UaG5B3GUZGK
Hash C: 19513fdc9da4fb72a4a05eb66917548d3c90ff94d5419e1f2363eea89dfee1dd
Hash D: $6$rounds=5000$saltsalt$dGhpcyBpcyBub3QgYSByZWFsIGhhc2g
```

> **Q2 (3 points):** For each hash (A through D), identify the hash type and explain how you determined it (what format clue did you use?).

Section 3: Cracking with John the Ripper (25 minutes)

John the Ripper ("john") is one of the oldest and most widely used password cracking tools. It supports many hash formats and runs well on CPU.

3.1 -- Preparing Your Wordlist

The rockyou.txt wordlist ships with Kali but is compressed. Decompress it:

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

If it is already decompressed, the command will say the file does not exist (the .gz version). Verify the file is ready:

```
wc -l /usr/share/wordlists/rockyou.txt
```

You should see approximately 14.3 million lines.

3.2 -- Creating Your Practice Hash File

Save these MD5 hashes to a file. Each line is the MD5 hash of a common password -- your job is to crack them:

```
cat << 'EOF' > practice_md5.txt
5f4dcc3b5aa765d61d8327deb882cf99
e10adc3949ba59abbe56e057f20f883e
0d107d09f5bbe40cade3de5c71e9e9b7
d8578edf8458ce06fbc5bb76a58c5ca4
21232f297a57a5a743894a0e4a801fc3
40be4e59b9a2a2b5dffb918c0e86b3d7
d0763edaa9d9bd2a9516280e9044d885
8621ffdbc5698829397d97767ac13db3
EOF
```

3.3 -- Running John

Crack the hashes with a dictionary attack:

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt practice_md5.txt
```

John will display cracked passwords as it finds them. This should complete quickly (under a minute) since these are common passwords.

3.4 -- Viewing Results

John stores cracked passwords in a "pot" file. View all cracked results:

```
john --format=raw-md5 --show practice_md5.txt
```

This displays each hash with its cracked plaintext.

> **Q3 (4 points):** How many of the 8 hashes did John crack? List any 4 of the cracked hash:password pairs from the `--show` output. How long did the cracking process take (check the output for timing)?

DELIVERABLE: Screenshot of your `john --show` output displaying the cracked passwords. The terminal command and results must both be visible.

Section 4: Cracking with Hashcat (25 minutes)

Hashcat is the industry standard for password cracking. It supports GPU acceleration (much faster than CPU) and offers more attack modes than John. In your VM, it will run in CPU mode, which is slower but functional for small hash files.

4.1 -- Hash Modes

Hashcat identifies hash types by mode number. Common modes:

Mode	Hash Type
-m 0	MD5
-m 1000	NTLM
-m 1400	SHA-256
-m 1800	SHA-512 crypt (Linux shadow)
-m 3200	bcrypt

You can see all modes with `hashcat --help` or search with:

```
hashcat --help | grep -i "md5"
```

4.2 -- Dictionary Attack

Before running hashcat, clear John's pot file so you can see fresh results (john and hashcat use separate pot files, but this is good practice):

```
hashcat -m 0 -a 0 practice_md5.txt /usr/share/wordlists/rockyou.txt --force
```

Flags explained:

- `-m 0`: MD5 hash mode
- `-a 0`: Straight dictionary attack (attack mode 0)
- `--force`: Allow CPU-only cracking in a VM (suppresses GPU warnings)

4.3 -- Understanding Hashcat Output

Hashcat displays progress information during cracking:

- **Speed:** Hashes per second (H/s). In a VM on CPU, expect thousands to millions per second for MD5. On a real GPU, this would be billions.
- **Progress:** How far through the wordlist it has processed.
- **Recovered:** How many hashes have been cracked.

View cracked results:

```
hashcat -m 0 practice_md5.txt --show
```

4.4 -- Comparing Tools

Both John and Hashcat cracked the same hashes. Key differences:

Feature	John the Ripper	Hashcat
Best for	CPU cracking, mixed formats	GPU cracking, large-scale
Speed (GPU)	Moderate	Fastest available
Output	Human-readable by default	Hash:password format
Config	Auto-detects formats	Requires mode number

> **Q4 (4 points):** Run hashcat against practice_md5.txt. What speed (H/s) did hashcat report? How does this compare to what a GPU could achieve? (Hint: check hashcat's benchmark output or documentation for typical GPU speeds on MD5.)

DELIVERABLE: Screenshot of your hashcat output showing the cracking run. The command, speed, and "Recovered" count must be visible.

Section 5: Advanced Techniques -- Rules and Masks (15 minutes)

A plain dictionary attack only cracks passwords that appear exactly in the wordlist. Many users modify dictionary words with predictable patterns: adding numbers, capitalizing the first letter, appending symbols. Rule-based attacks exploit these patterns.

5.1 -- Why Dictionaries Alone Fail

Save these hashes to a new file. These represent passwords that are NOT simple dictionary words:

```
cat << 'EOF' > challenge_md5.txt
541964299fdf7446cd43adf5ea2c9cd8
161ebd7d45089b3446ee4e0d86dbcf92
3db2e760dec0dddb9e83cacd6d3a16ba
b56e0b4ea4962283bee762525c2d490f
a385e2a982311b1c4a6979746fe85350
EOF
```

First, try a plain dictionary attack:

```
hashcat -m 0 -a 0 challenge_md5.txt /usr/share/wordlists/rockyou.txt --force
```

Some of these may crack, but others will resist a straight dictionary attack.

5.2 -- Rule-Based Attacks

Rules tell hashcat to mutate each dictionary word: capitalize it, append numbers, substitute characters, reverse it, and more. Hashcat ships with several rule files:

```
ls /usr/share/hashcat/rules/
```

The `best64.rule` file contains 64 of the most effective mutations. Run the same attack with rules:

```
hashcat -m 0 -a 0 challenge_md5.txt /usr/share/wordlists/rockyou.txt -r
/usr/share/hashcat/rules/best64.rule --force
```

Check results:

```
hashcat -m 0 challenge_md5.txt --show
```

5.3 -- Mask Attacks

Mask attacks are useful when you know the password pattern. For example, to crack a 4-digit PIN:

```
hashcat -m 0 -a 3 hash_file.txt ?d?d?d?d --force
```

Mask characters:

- `?d` = digit (0-9)
- `?l` = lowercase letter (a-z)
- `?u` = uppercase letter (A-Z)
- `?s` = special character
- `?a` = any printable character

A mask like `?u?l?l?l?l?d?d?d` would match patterns like "Admin123" or "Hello456".

> **Q5 (4 points):** Run the rule-based attack against challenge_md5.txt. How many of the 5 hashes cracked with the plain dictionary? How many cracked after adding the best64.rule? List the cracked passwords.

> **Q6 (4 points):** In your own words, explain why a password like "Welcome1" would be cracked by a rule-based attack but might survive a plain dictionary attack. What rule transformation would catch it?

DELIVERABLE: Screenshot of your hashcat output from the rule-based attack showing cracked results.

Section 6: Enterprise Scenario -- BrightStar Technologies Breach (30 minutes)

This section simulates a real penetration testing engagement. Apply everything you have learned to crack a set of stolen password hashes and analyze the results.

6.1 -- The Scenario

During a penetration test of BrightStar Technologies, you extracted a database backup containing employee password hashes. The application uses unsalted SHA-256 hashing (a common but insecure practice). The database included usernames and job titles alongside the hashes.

Your task: crack as many passwords as possible and write a password audit report assessing the organization's password practices.

6.2 -- The Hash Dump

Save the following to a file. Each line contains a username and their SHA-256 password hash:

```
cat << 'EOF' > enterprise_hashes.txt
jsmith:19513fdc9da4fb72a4a05eb66917548d3c90ff94d5419e1f2363eea89dfee1dd
admin:7e19e31ae82d749034fc921f777f717ba5b57c6add9add889eb536ac6effcde0
sclark:48486e1514e842346ff405b1e45f44059ae82619f2306f99d0940dcb386e91f7
mlopez:17bb571b42297c563813eb7bfec91a19d2c724612e02871a64eb90364759ca13
kpatel:a9c43be948c5cabd56ef2bacffb77cdaa5eec49dd5eb0cc4129cf3eda5f0e74c
jdoe:6ca13d52ca70c883e0f0bb101e425a89e8624de51db2d2392593af6a84118090
rjones:203b70b5ae883932161bbd0bded9357e763e63afce98b16230be33f0b94c2cc5
akim:fc613b4dfd6736a7bd268c8a0e74ed0d1c04a959f59dd74ef2874983fd443fc9
backup_svc:6d3766bc4f39cfc5c6cb7a249e3314b1848887ee5d30fb25caf9d86da05c9bf5
svc_monitor:e3f07fb7ffa76db024342eb4b9d237bd1395d2ee51717d33d42e3a08ec01ad19
EOF
```

Employee directory (for your report):

Username	Role
jsmith	IT Manager
admin	System Administrator
sclark	CEO
mlopez	HR Director
kpatel	Developer
jdoe	Intern
rjones	CFO
akim	Database Administrator
backup_svc	Backup Service Account
svc_monitor	Monitoring Service Account

6.3 -- Cracking the Hashes

Start with a dictionary attack using hashcat. Note the `--username` flag, which tells hashcat the file contains `user:hash` format:

```
hashcat -m 1400 -a 0 enterprise_hashes.txt /usr/share/wordlists/rockyou.txt --username --force
```

After the dictionary attack completes, try adding rules:

```
hashcat -m 1400 -a 0 enterprise_hashes.txt /usr/share/wordlists/rockyou.txt --username -r /usr/share/hashcat/rules/best64.rule --force
```

View all cracked results:

```
hashcat -m 1400 enterprise_hashes.txt --username --show
```

6.4 -- Analyzing Your Results

Review which accounts were cracked and which resisted. Consider:

- Which roles had the weakest passwords?
- Are service accounts using predictable naming patterns?
- Do any cracked passwords follow patterns (seasonal words, company name, simple dictionary words)?
- What does this tell you about the organization's password policy (or lack thereof)?

> **Q7 (4 points):** How many of the 10 enterprise hashes did you crack? List each cracked account in the format `username:password`. Which accounts resisted cracking?

> **Q8 (4 points):** Look at the cracked passwords across all roles. Identify at least two patterns you observe (e.g., dictionary words, seasonal patterns, predictable service account names). What does this tell you about BrightStar's password policy?

DELIVERABLE: Screenshot of your `hashcat --show` output for the enterprise hashes, showing cracked username#password entries.

Challenge Extensions (Advanced Students)

If you completed the core lab with time to spare, tackle these challenges. Document your solutions.

Challenge A: NTLM Hashes (Active Directory Style)

In Active Directory environments, passwords are stored as NTLM hashes (MD4 of the UTF-16LE encoded password). On your Kali VM, generate an NTLM hash:

```
echo -n "password" | iconv -t UTF-16LE | openssl dgst -md4 | awk '{print $2}'
```

Generate NTLM hashes for 3 passwords of your choice, save them to a file, and crack them with hashcat using mode 1000:

```
hashcat -m 1000 -a 0 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt --force
```

Document the hashes you generated, the commands you used, and the cracking results.

Challenge B: Custom Wordlist with CeWL

CeWL is a tool that scrapes a website and builds a custom wordlist from its content. Company-specific terms often appear in employee passwords. Run CeWL against a website (the class VPS or any public site):

```
cewl http://97.107.132.247 -d 2 -m 5 -w custom_wordlist.txt
```

How many words did CeWL generate? Try using this custom wordlist (with rules) against the enterprise hashes. Did it crack anything rockyou.txt missed?

Challenge C: Mask Attack -- Brute Force a PIN

Create an MD5 hash of a 6-digit number (your choice), then use hashcat's mask attack to brute force it:

```
# Create your hash (pick any 6 digits)
echo -n "847291" | md5sum | awk '{print $1}' > pin_hash.txt

# Crack it with a mask
hashcat -m 0 -a 3 pin_hash.txt ?d?d?d?d?d?d --force
```

How long did it take? How many combinations did hashcat try? Calculate the keyspace (10^6 = 1,000,000 combinations).

Deliverables

Submit the following to Blackboard:

Part A: Checkpoint Questions (30 points)

Answer all 8 checkpoint questions with specific commands and output:

- Q1: 3 points (hashing fundamentals)
- Q2: 3 points (hash identification)
- Q3: 4 points (John the Ripper results)

- Q4: 4 points (Hashcat results and speed)
- Q5: 4 points (rule-based attack results)
- Q6: 4 points (explain rule-based attacks)
- Q7: 4 points (enterprise cracking results)
- Q8: 4 points (password pattern analysis)

Part B: Evidence Screenshots (20 points)

These screenshots were captured inline during the lab:

1. **John the Ripper output** (5 points): `john --show` results from Section 3
2. **Hashcat output** (5 points): Hashcat cracking run from Section 4
3. **Rule-based attack** (5 points): Hashcat rule-based results from Section 5
4. **Enterprise scenario** (5 points): `hashcat --show` results from Section 6

Part C: Password Audit Report (35 points)

Write a professional password audit report for BrightStar Technologies using this structure:

Report Header (5 points): Your name, date, client name (BrightStar Technologies), assessment scope.

Methodology (5 points): Tools used, wordlists, attack types (dictionary, rule-based, mask). Be specific -- name the exact wordlist and rule file.

Findings (15 points): For each cracked account, document:

Field	Content
Username	[username]
Role	[from employee directory]
Cracked Password	[plaintext]
Time to Crack	[approximate]
Weakness	[why this password is weak -- e.g., dictionary word, common pattern]

Also document which accounts were NOT cracked and what that suggests about their password strength.

Risk Assessment (5 points): Overall assessment of the organization's password practices. What percentage of passwords were cracked? What patterns exist? What is the business impact (consider the roles of the people with weak passwords)?

Recommendations (5 points): Specific, actionable recommendations for improving password security. Consider: minimum length, complexity requirements, password managers, multi-factor authentication, service account policies.

Part D: Reflection Questions (15 points)

> **R1 (8 points):** You cracked passwords for employees at multiple levels of the organization (intern to CEO). Discuss the security implications of each cracked role. If you were an attacker, which account would you target first and why? How would you use that initial access to move further into the organization?

> **R2 (7 points):** Compare the cracking speed you observed in your VM (CPU-only) with what a dedicated cracking rig with multiple GPUs could achieve. A single RTX 4090 can compute approximately 164 billion MD5 hashes per second. How does this change the practical security of different hash types? Which hash types from Section 1.4 would still be resistant, and why?

Pre-Submission Checklist

Before submitting, verify:

- [ ] All 8 checkpoint questions answered with actual commands and output (not hypothetical)
- [ ] 4 screenshots captured (john output, hashcat output, rule-based results, enterprise results)
- [ ] Part C report includes specific cracked passwords and their weaknesses (not generic)
- [ ] Part C recommendations are specific and actionable (not "use better passwords")
- [ ] Your name and date appear in the Part C report header
- [ ] All files attached to Blackboard submission (not links to Google Docs or SharePoint)

Preparation for Lab 10

In the next lab, you will work with wireless network security. You will use specialized hardware (USB WiFi adapters) to capture wireless traffic and crack WPA2 handshakes -- applying the same cracking techniques you learned today, but against network authentication instead of database passwords.

To prepare:

- Review the concept of WPA2 authentication (the 4-way handshake)
- Think about how the password cracking workflow (capture hash, run dictionary attack) applies to WiFi passwords
- No hardware setup is needed -- the instructor will provide WiFi adapters in class

---

Quick Reference

Tool Commands

Task	John the Ripper	Hashcat
Crack MD5	`john --format=raw-md5 --wordlist=FILE HASHES`	`hashcat -m 0 -a 0 HASHES FILE --force`
Crack SHA-256	`john --format=raw-sha256 --wordlist=FILE HASHES`	`hashcat -m 1400 -a 0 HASHES FILE --force`
Crack NTLM	`john --format=nt --wordlist=FILE HASHES`	`hashcat -m 1000 -a 0 HASHES FILE --force`
Show results	`john --show HASHES`	`hashcat -m MODE HASHES --show`
With rules	`john --rules --wordlist=FILE HASHES`	`hashcat -m MODE -a 0 HASHES FILE -r RULES --force`
With username	N/A (auto-detected)	Add `--username` flag

Hashcat Modes

Mode	Hash Type
0	MD5
100	SHA-1
1000	NTLM
1400	SHA-256
1800	SHA-512 crypt
3200	bcrypt

Hashcat Mask Characters

Mask	Matches
`?d`	Digit (0-9)
`?l`	Lowercase (a-z)
`?u`	Uppercase (A-Z)
`?s`	Special characters
`?a`	Any printable character

Common Wordlists (Kali)

Wordlist	Path	Size
rockyou.txt	`/usr/share/wordlists/rockyou.txt`	~14.3 million
fasttrack.txt	`/usr/share/wordlists/fasttrack.txt`	222 words
dirb common	`/usr/share/dirb/wordlists/common.txt`	~4,600 words

Common Rule Files (Hashcat)

Rule File	Path	Description
best64.rule	`/usr/share/hashcat/rules/best64.rule`	64 most effective rules
rockyou-30000.rule	`/usr/share/hashcat/rules/rockyou-30000.rule`	30,000 rules from analysis
d3ad0ne.rule	`/usr/share/hashcat/rules/d3ad0ne.rule`	Comprehensive rule set

Resources

- Hashcat Wiki -- Official documentation and examples
- John the Ripper Documentation -- Usage and format support
- CrackStation -- Online hash lookup (educational reference)
- Have I Been Pwned -- Check if credentials appeared in known breaches

---

Lab 9 Complete. You now understand how password cracking works in practice -- from hash identification through tool selection to enterprise-scale password auditing. These skills are central to every penetration testing engagement.